

Vector space databases and Clustering

Hoa Dinh Nguyen, PhD

hoand@ptit.edu.vn

Vector Space Model for Multimedia Retrieval

- ▶ Each record (document, image, audio clip) → N-dimensional feature vector $x = (x_1, \dots, x_N)$
- ▶ Features: TF-IDF weights (text), color histogram bins (image), MFCC stats (audio)
- ▶ Query $q = (q_1, \dots, q_N)$ in the same feature space
- ▶ Retrieval: rank all records by similarity $\text{sim}(d, q)$ — highest first
- ▶ Binary or real-valued features; vectors must be comparable (same feature definition)
- ▶ Challenge: different vector lengths (different docs) → normalization required



Vector space database

- ▶ Assumption: Database is presented in vectors of features of size N.

$$D_i = [T_{i1}, T_{i2}, \dots, T_{ik}, \dots, T_{iN}]$$

$$Q_j = [Q_{j1}, Q_{j2}, \dots, Q_{jk}, \dots, Q_{jN}]$$

- ▶ T_{ik} is the value of feature k^{th} in record i ,
- ▶ Q_{jk} is the value of feature k^{th} in query j ,
- ▶ T_{ik} and Q_{jk} can have binary values $\{1, 0\}$ or any numerical values.
- ▶ Questions:
 - ▶ How can we compute the similarity between the two vectors with a small N?
 - ▶ How can we compute the similarity between the two records having different vector lengths?

Vector space database

- ▶ Query process is based on the similarity between the content of the Query and that of the database
- ▶ The similarity between the data D_i and the query Q_j can be calculated as the inner product of the two vectors.

$$S(D_i, Q_j) = \sum_{k=1}^N d_{ik} \cdot q_{jk}$$

Vector space database

- ▶ Question:
 - ▶ The inner product is good for comparing vectors with the same length.
 - ▶ How can we consider the lengths of different vectors as a factor in similarity computation?

Vector space database

- ▶ Normalized similarity: take into account the different lengths of the vectors in similarity formula:

$$S(D_i, Q_j) = \frac{\sum_{k=1}^N d_{ik} \cdot q_{jk}}{\sqrt{\sum_{k=1}^N d_{ik}^2 \cdot \sum_{k=1}^N q_{jk}^2}}$$

- ▶ Query results are sorted in descending order of similarity.

Inner Product and Cosine Similarity

- ▶ Inner product: $\text{sim}(d, q) = d \cdot q = \sum_i d_i q_i$ — favors long vectors (many non-zero terms)
- ▶ Vector length: $|d| = \sqrt{(\sum_i d_i^2)}$; normalized similarity (cosine):
 - ▶ $\text{sim}(d, q) = (d \cdot q) / (|d| \times |q|)$ — range $[-1, 1]$, typically $[0, 1]$ for non-negative features
- ▶ Cosine measures angle between vectors — independent of document length
- ▶ Equivalent to inner product on L2-normalized unit vectors: $d/|d|, q/|q|$
- ▶ Standard in text IR and CBIR when histograms are normalized



Vector space database

▶ Example: Given 4 database vectors with the same size as

▶ $D1=[0.2,0.3,0.5,0.0,0.1]$ $D2=[0.3,0.3,0.2,0.1,0.2]$
 $D3=[0.2,0.2,0.1,0.2,0.3]$ $D4=[0.1,0.1,0.3,0.1,0.0]$

Given following query

▶ $Q=[0.2,0,0.1,0,0.2]$

- ▶ (a) Compute dot product $S(D_i, Q)$ for $i=1..4$
- ▶ (b) Compute $|D_i|$ and $|Q|$; then cosine similarity for each D_i
- ▶ (c) Rank documents by dot product vs by cosine — does ranking change?
- ▶ (d) Which measure is appropriate when vectors are normalized?

Vector space database

- ▶ Disadvantages:

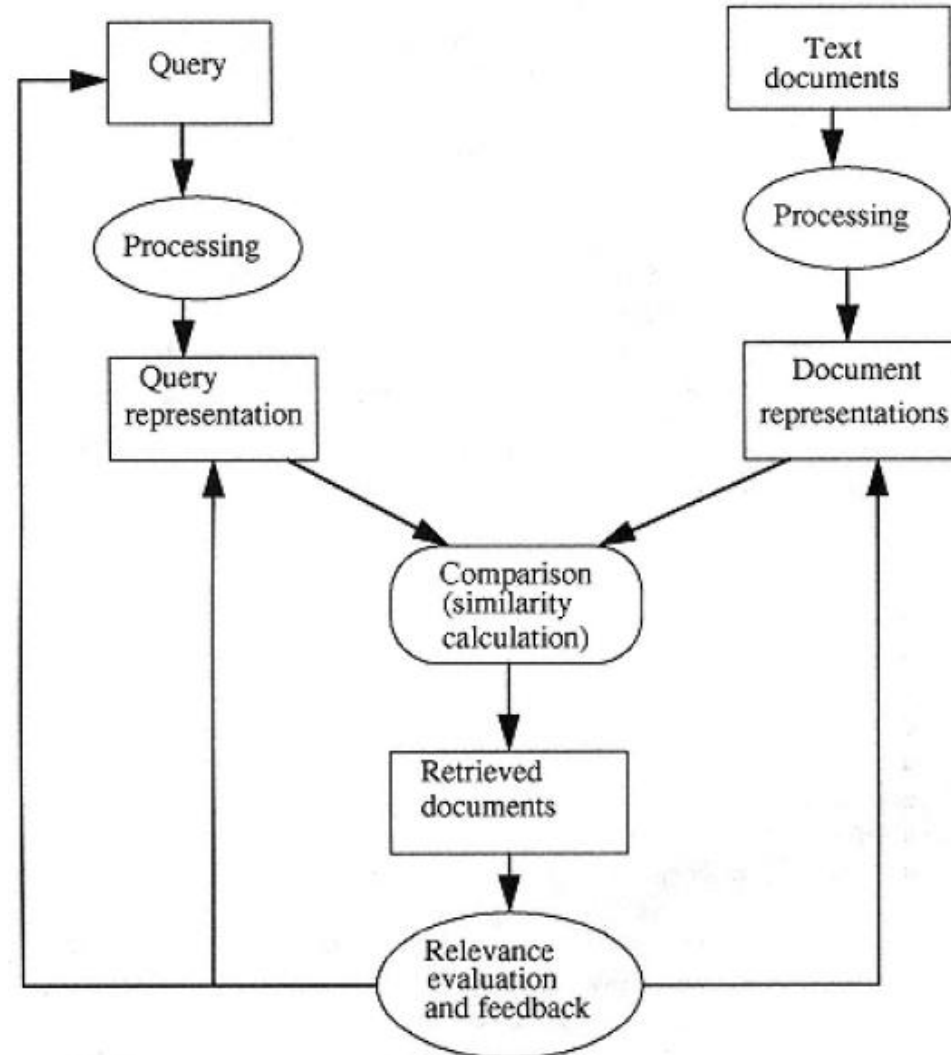
- ▶ The vector-space query model does not take into account the difference in meaning between different attributes
- ▶ Only works well with small sized vector databases and queries.

Relevance Feedback — Motivation

- ▶ Problem: initial query vector may not capture user intent (semantic gap)
- ▶ User marks returned results as relevant (R) or non-relevant (NR)
- ▶ System adjusts query (and optionally document vectors) to improve next retrieval
- ▶ Rocchio algorithm: move query toward centroid of relevant docs, away from non-relevant
- ▶ Used in text IR, image CBIR, and multimedia search interfaces
- ▶ Iterative: user can refine over multiple feedback rounds

Vector space database

► Queries with feedback



Vector space database

- ▶ Query with feedback system
 - ▶ Adjust query vectors to improve search results
 - ▶ Features appearing in relevant records but not in the query will be added to the query vector with some small values.
 - ▶ Features appearing in both relevant records and the query will have their values increased in the query vector.
 - ▶ Features appearing in both irrelevant records and the query will be removed from the query vector, or have their values reduced in the query vector
 - ▶ Perform a search with a new query after adjustment.

Vector space database

- ▶ Standard Rocchio query update :

$$Q_{j+1} = \alpha \cdot Q_j + \beta \cdot \sum_{D^k \in Rel} D^k - \gamma \cdot \sum_{D^t \in NonRel} D^t$$

where:

- ▶ *Rel*: set of records related to the query content among the returned results
- ▶ *NonRel*: set of records unrelated to the query content among the returned results
- ▶ Typical: $\alpha = 1$, $\beta = 0.75$, $\gamma = 0.15$ (tunable)
- ▶ Update query vector toward successful queries
- ▶ Limitation: assumes linear feature space; less effective for deep embedding retrieval

Vector space database

- ▶ Adjust the database vectors – to influence other queries (of other users).
 - ▶ Features that are in the query, but not in the record that are considered relevant are added to the record vector with a value initialized with a certain value
 - ▶ The value of features that are both in the query and the relevant record is increased.
 - ▶ The value of features that are not in the query but are in the record is reduced.
 - ▶ Make a new search with the same query as the previous search.
- This method won't be useful if the new query is too different from the previous one used to change the database vectors.

Clustering

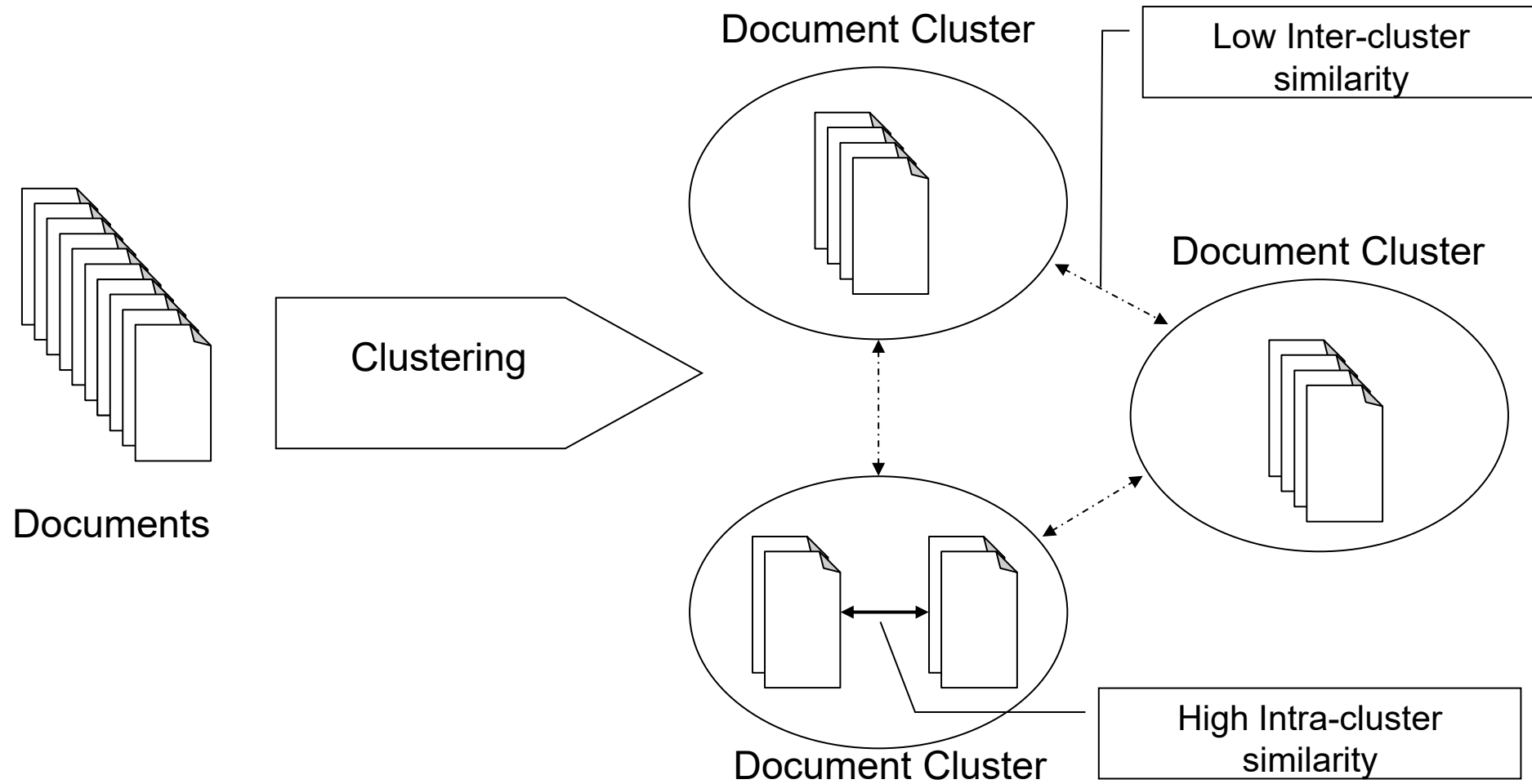
- ▶ Clustering is the process of grouping records that are similar in content into clusters according to some rules:
 - ▶ Records in the same cluster must have the same contents
 - ▶ Records in different clusters have different contents.

Clustering — Definitions and Goals

- ▶ Goal: partition n records into c groups (clusters) without class labels (unsupervised)
- ▶ High intra-cluster similarity: records within same cluster are alike
- ▶ Low inter-cluster similarity: different clusters are well separated
- ▶ Applications in MMDB: organize image/audio features, browse by cluster, speed up k-NN
- ▶ Cluster centroids can serve as index entry points
- ▶ Quality metrics: silhouette score, within-cluster sum of squares (WCSS)

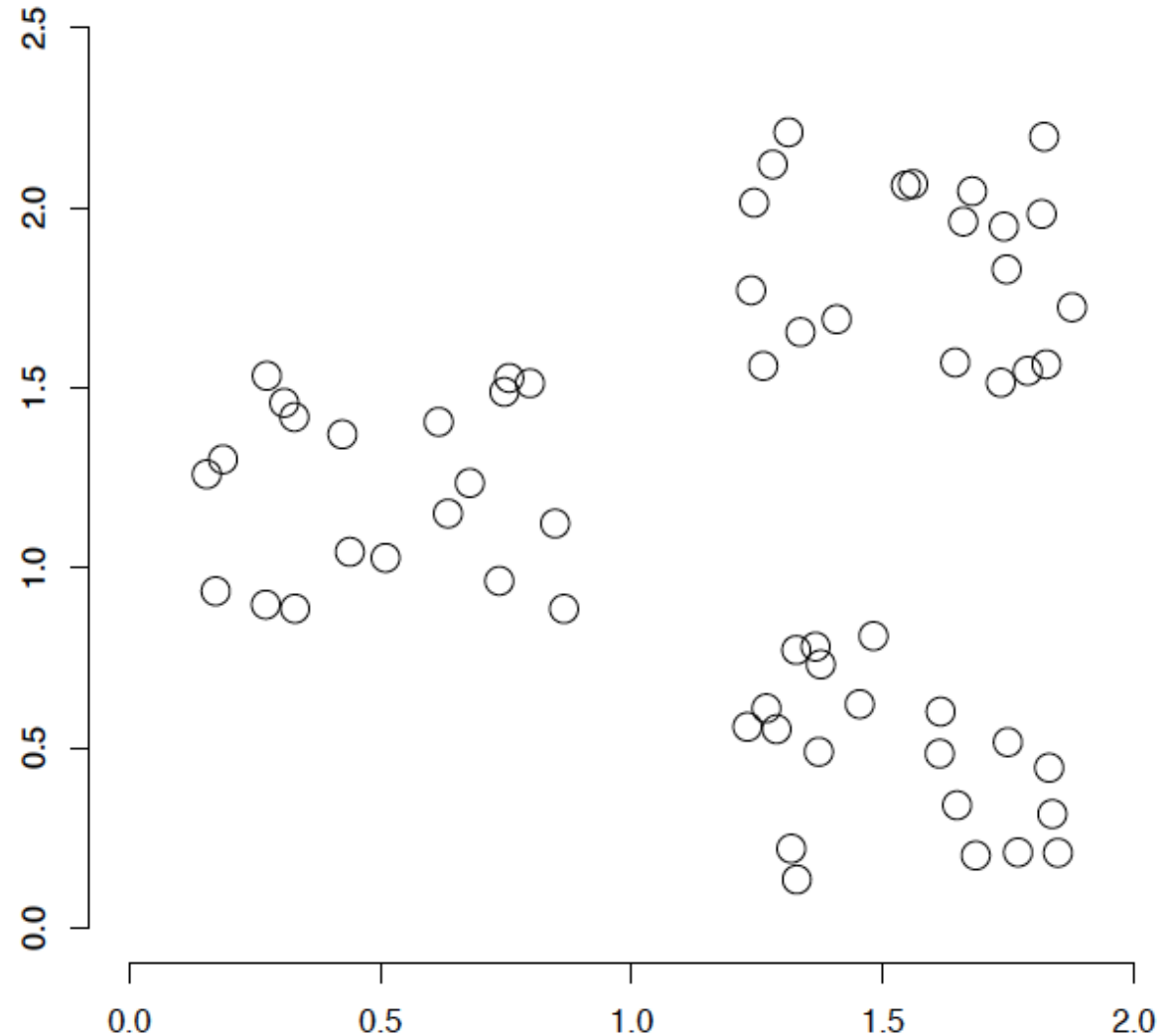


Clustering



Example of a dataset with a clustered structure

- Identify an algorithm to divide this dataset into different clusters.

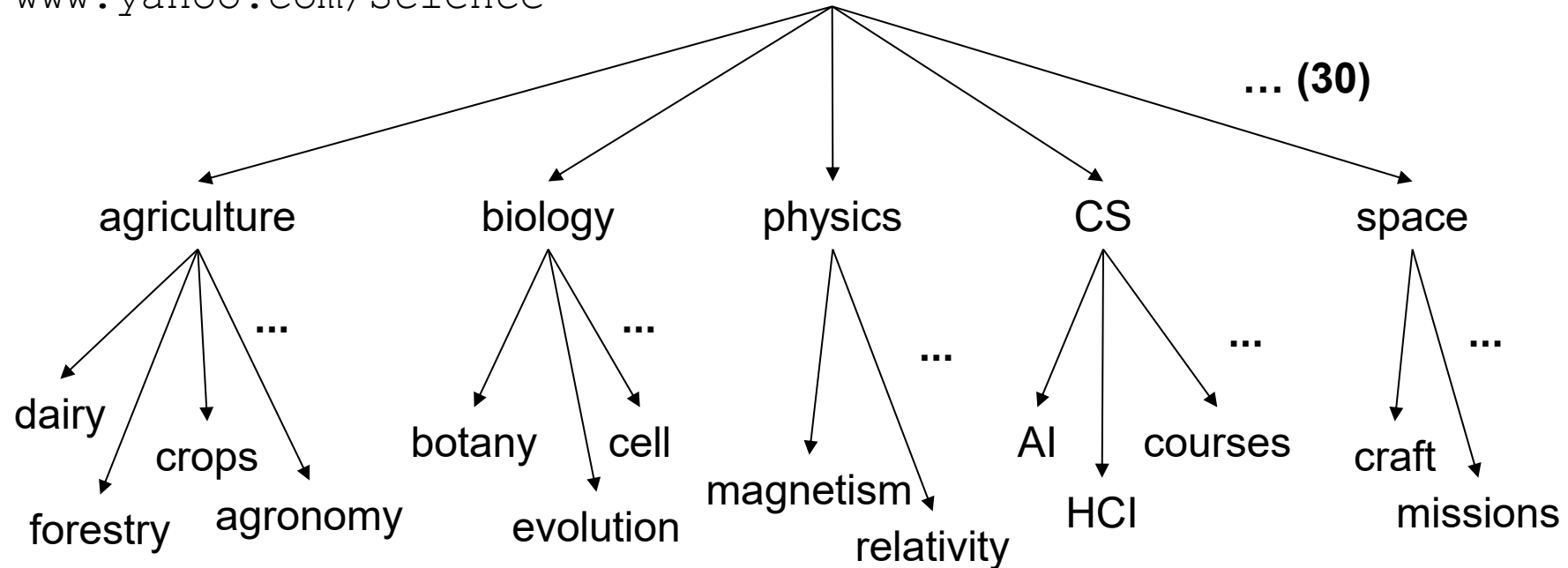


Similarity measures

- ▶ Similarity score between 2 records
- ▶ Distance score between 2 records
 - ▶ L_p -norm: $L_p(D_i, D_j) = \sqrt[p]{\sum_{k=1}^N |d_{ik} - d_{jk}|^p}$
 - ▶ L_1 -norm (Manhattan): $L_1(D_i, D_j) = \sum_{k=1}^N |d_{ik} - d_{jk}|$
 - ▶ L_2 -norm (Euclidean): $L_2(D_i, D_j) = \sqrt{\sum_{k=1}^N |d_{ik} - d_{jk}|^2}$
 - ▶ Cosine distance: $1 - \cos(x, y)$, for sparse high-dim vectors
- ▶ Inner product similarity vs distance: $sim = -d$ for normalized vectors
- ▶ Choice depends on feature type: L_1 robust for histograms; L_2 for geometric space

Example of Yahoo's clustered data structure

www.yahoo.com/Science



Clustering Methods — Overview

- ▶ Hierarchical (pair-wise): agglomerative merge until c clusters — no preset c
- ▶ Random selection: online threshold-based assignment — order-dependent
- ▶ K-means (hard c -means): each point belongs to exactly one of c clusters
- ▶ Fuzzy c -means (FCM): soft membership — point belongs partially to multiple clusters
- ▶ Selection guide: K-means/FCM for large MM feature sets; hierarchical for small n
- ▶ c = number of clusters (user-specified for K-means and FCM)



Clustering based on pair-wise comparison

- ▶ Each record is considered as a cluster of its own
- ▶ Forming clusters from pairs of records or pairs of existing clusters
- ▶ A new cluster is formed based on the most similar pair of records or pair of clusters.
 - ▶ Criteria for comparing two pairs of record clusters:
 - ▶ Based on the highest similarity among all record pairs between two clusters
 - ▶ Based on the greatest difference among all record pairs between two clusters
 - ▶ Based on the average of the similarity of all record pairs between the two clusters
- ▶ The process is repeated until all records are located in a certain cluster

Criteria for clustering

- ▶ Criteria based on the most similar by pair

$$\textit{sim}(c_i, c_j) = \max_{x \in c_i, y \in c_j} \textit{sim}(x, y)$$

- ▶ Criteria based on the most different pairs

$$\textit{sim}(c_i, c_j) = \min_{x \in c_i, y \in c_j} \textit{sim}(x, y)$$

Clustering based on random selection

- ▶ The first record is randomly chosen as a cluster of itself
- ▶ Randomly select another record, calculate the similarity (or distance) between that record and the existing record clusters.
- ▶ If the new record is similar to an existing cluster (based on the comparison threshold λ) it will be included in the cluster, otherwise it becomes a new cluster.
- ▶ The procedure is repeated until all records are reviewed.

K-means algorithm

- ▶ Records represented by feature vectors
- ▶ Initialize K group center points in the data space
- ▶ The records are selected in turn into one of the K groups corresponding to the shortest distance to the center points.
- ▶ The new positions of K center points are reset as the means of the K newly formed data groups
- ▶ The cycle is reinserted until the
 - ▶ No more changes in the clusters of records
 - ▶ The centers of the groups do not change
 - ▶ Exceed a given maximum number of loops.

K-Means Algorithm — Hard c-Means

- ▶ Input: n vectors $x_1 \dots x_n$, number of clusters c , distance metric (usually L_2)
- ▶ Step 1: Initialize c centroids $v_1 \dots v_c$ (random or k-means++)
- ▶ Step 2 (Assign): each $x_i \rightarrow \text{cluster } k = \operatorname{argmin}_j d(x_i, v_j)$
- ▶ Step 3 (Update): $v_k = \text{mean of all points assigned to cluster } k$
- ▶ Step 4: Repeat Steps 2–3 until centroids unchanged or max iterations
- ▶ Output: hard partition — each point in exactly one cluster; centroids v_k



Fuzzy c-Means (FCM) — Soft c-Means Clustering

- ▶ Extension of c-means: each point x_i has membership degree $u_{ik} \in [0,1]$ to cluster k
- ▶ Constraint: $\sum_{k=1}^c u_{ik} = 1$ for each point i (fully partitioned fuzzily)
- ▶ Objective function (minimize): $J_m = \sum_i \sum_k u_{ik}^m ||x_i - v_k||^2$
 - ▶ $m > 1$: fuzzifier ($m = 2$ most common); $m \rightarrow 1$ approaches hard K-means
- ▶ Membership update:

$$u_{ik} = 1 / \sum_{j=1}^c \left(\frac{||x_i - v_k||}{||x_i - v_j||} \right)^{\frac{2}{m-1}}$$

- ▶ Centroid update: $v_k = \frac{\sum_i u_{ik}^m \cdot x_i}{\sum_i u_{ik}^m}$
- ▶ Iterate until J_m stabilizes or max iterations



K-Means vs Fuzzy c-Means — Comparison

- ▶ K-means: hard assignment ($u_{ik} \in \{0, 1\}$), faster, simpler, local minima sensitive
- ▶ FCM: soft assignment — handles overlapping clusters, boundary points partially shared
- ▶ K-means: use when clusters are compact, well-separated (e.g., genre groups in audio)
- ▶ FCM: use when data has gradual transitions (image regions, document topics overlap)
- ▶ Both require choosing c ; FCM adds parameter m (fuzziness)
- ▶ MMDB: store cluster id (K-means) or top membership vector (FCM) as metadata per record



Querying data in a clustering model

- ▶ The cluster of records closest to the query is determined based on the distance between the query and the center of the cluster
- ▶ If the cluster of records contains a small number of records, all records in the cluster will be the result of the search
- ▶ If the cluster contains a large number of records: only 1 or k records that are most similar to the query will be the result.

Querying Clustered Vector Data

- ▶ Step 1: compute distance from query q to all c cluster centroids
- ▶ Step 2: select nearest cluster(s) — probe only those clusters (filter step)
- ▶ Step 3: if cluster small $\rightarrow$ return all members; if large $\rightarrow$ top- k by $\text{sim}(q, d)$
- ▶ Multi-probe: search 2–3 nearest clusters to avoid missing border points
- ▶ FCM extension: rank by $\sum_k u_{ik} \cdot \text{sim}(q, v_k)$ or assign to cluster with $\max u_{ik}$
- ▶ Example: cluster color histograms for faster image search



Summary — Vector Space DB and Clustering

- ▶ Vector space model: features as vectors; cosine similarity for length-normalized retrieval
- ▶ Rocchio feedback: refine queries using relevant/non-relevant judgments
- ▶ Clustering organizes MM records for browsing and efficient filtered search
- ▶ K-means (hard c-means): fast partition; FCM (fuzzy c-means): soft overlapping clusters
- ▶ Hierarchical and random methods for small data or streaming scenarios
- ▶ Cluster-based query: search nearest centroid first, then refine within cluster

